Symbolic Execution is (Not Quite) All You Need

Sophie Smithburg

***Abstract*—We present a technique for extracting verified dispatch automata from ELF binaries via symbolic execution and partition refinement. The extracted labeled transition system is bisimilar to the concrete dispatch behavior, proved in Lean 4 with no admitted axioms. Guards and updates are expressed in the quantifier-free bitvector fragment of SMT-LIB, enabling automated vulnerability detection via Datalog queries. In a blind trial, a 119-line Soufflé program correctly identified functions corresponding to CVE-2020-36366 through CVE-2020-36375 in mjs v1.20.1 from anonymized dispatch facts alone.**

***Index Terms*—symbolic execution, formal verification, labeled transition systems, partition refinement, LangSec, Lean 4, Datalog**

I. Introduction

Specifications that admit formal reasoning—labeled transition systems, rewrite semantics, mechanized grammars—are the exception, not the norm. Most software is specified, if at all, in natural language that is too imprecise to prove anything about. The Python Language Reference itself notes that "if you were coming from Mars and tried to re-implement Python from this document alone, you might have to guess things and in fact you would probably end up implementing quite a different language" [1]. Implementations are what we actually get. But you can't prove things about implementations directly—you can about specs. So in order to have an effective ratchet in the sense of Metzger's LangSec keynote [2]—addressing the specification-implementation gap [3]—we want to be able to get specs from implementations.

We can model a programming language executing on a host machine as a host labeled transition system H [4] which, after having been fed $\mathcal{I}$, the implementation, simulates G—the PL is G, its implementation is $\mathcal{I}$, and the host machine is H. To prove a recovered G' faithfully captures G, we would need G in a formalism that shares the same proof basis as G'—but in practice, this is never the case. So what we do instead is prove that G' simulates $H_{\mathcal{I}}$, by way of projecting away dispatch-irrelevant state and transitions.

We have implemented a technique and have an early proof of concept of a system that, by way of symbolic execution and Paige-Tarjan partition refinement [5], can extract a strongly bisimilar symbolic labeled transition system from an ELF binary—guards and updates expressed in the quantifier-free bitvector fragment of SMT-LIB. We have successfully used this to generate vulnerability detection queries in Soufflé [6], a Datalog dialect oriented towards static analysis. In a blind trial, Claude Opus 4.6 was given only two toy synthetic examples of a vulnerability class, wrote a 119-line Soufflé program that distinguished bounded from unbounded recursion, and when applied blind to anonymized facts from mjs v1.20.1, correctly identified functions corresponding to CVE-2020-36366 through CVE-2020-36375. We also used the technique to confirm the absence of certain terminal emulator vulnerability classes in libtsm, by passing a description of the vulnerability classes along with the extracted LTS to an LLM agent.

This paper and its Lean 4 mechanization were developed with extensive use of Claude Code and ChatGPT, some use of Codex CLI, and a smattering of other LLM tools in earlier prototypes.

Contributions.

1) A technique for extracting dispatch automata from ELF binaries via symbolic execution and Paige-Tarjan partition refinement, producing a finite LTS bisimilar to the concrete system—not just the dispatch structure but the full data transformation—with guards and updates in QF_UFBV.
2) Proofs mechanized in Lean 4 with zero `sorry` and zero admitted axioms.
3) A convergence guarantee: every finite binary has finitely many dispatch equivalence classes, so extraction always terminates.
4) A converter from LTS JSON to Soufflé fact files, enabling automated vulnerability detection via Datalog queries over symbolic guard expressions.
5) A blind CVE confirmation: a 119-line Soufflé program, written from toy examples alone, correctly identifies CVE-2020-36366 through CVE-2020-36375 in mjs v1.20.1 from anonymized dispatch facts.
6) Evidence that symbolic guards are the key differentiator over existing binary analysis tools: structural cycle detection cannot distinguish bounded from unbounded recursion; our symbolic guard expressions enable the distinction.

II. Background

The Language-Theoretic Security thesis [3] holds that computational power is a dimension of the attack surface: an implementation that exceeds the computational requirements of its specification should be considered broken. When an implementation admits more computational power than its specification requires, the excess constitutes a weird machine—an unintended computational substrate available to an attacker.

We introduce the *dispatch automaton*: the control-flow automaton (CFA) [7] of a program's dispatch loop, projected to retain only dispatch-relevant state and with dispatch-irrelevant transitions elided. A parser's dispatch automaton is the grammar it implements. An interpreter's dispatch automaton is its opcode dispatch table. A terminal emulator's dispatch automaton is its VT state machine.

The dispatch automaton's computational class determines what the implementation can do beyond its specification. We adopt the standard hierarchy from automata theory: finite languages (fixed command dispatch), regular (flat input processing), visibly pushdown [8] (recursive descent with bracketed call/return), and context-sensitive or higher (accumulated state influencing dispatch). The classification boundary that matters for LangSec is: if a regular-spec parser has a context-sensitive dispatch automaton, the excess computational power is a potential weird machine.

For systems designed for human use, the dispatch automaton is almost always regular or visibly pushdown—humans design in finite terms. The implementation is a finite specification with engineering noise. Our tool recovers the signal.

III. What We Extract

The output of our pipeline is a finite labeled transition system (LTS). We conjecture that this LTS is strongly bisimilar [9] to the concrete dispatch behavior of the input binary, in the sense of van Glabbeek's spectrum [10]. The argument: the extracted LTS contains no internal (τ) transitions—all computation between dispatch points is composed into direct labeled transitions during enumeration, and in the absence of τ-transitions, bisimulation and strong bisimulation coincide. Formalizing this in Lean requires one additional lemma (that deterministic τ-elimination preserves bisimulation), which is in progress.

We note that the extracted LTS may be slightly larger than the pure dispatch specification: stack frame manipulation and other host-level implementation detail are preserved in transition effects, not hidden. We are bisimulating the guest-level system plus a bit of the host-level system at the same time. This may be addressable with better ABI awareness in a future version; for a work-in-progress report, a slightly too-large LTS with a bit of extra detail is acceptable.

Guards and updates are expressed in the quantifier-free bitvector fragment of SMT-LIB (QF_UFBV), with uninterpreted functions for memory load and store operations. Expressions are simplified via CVC5 [11] and deduplicated into a shared symbol DAG, reducing LTS files from 45 MB to 7 KB in representative cases.

Our algorithm converges by way of ensuring that we have reached a fixed point of composing the dispatch body with itself. A finite binary has a finite number of branch guards; with k guards and three-valued evaluation (true, false, or unknown), there are at most 3^k possible PC-signatures. Since there are finitely many possible signatures, this fixed point is always reached. Moreover, we conjecture that no finite program can have a counting dispatch automaton: a finite binary has finitely many places to dispatch and finitely many ways to dispatch—the counter can be unbounded in the data, but the program's response to it can only have finitely many distinct cases, because there are only finitely many branches in the code. Convergence is therefore guaranteed for all finite binaries; the only limit is scalability.

We get bisimulation precisely when the system is visibly pushdown or regular or finite in its dispatch automaton—because the precise criterion that pushes a system from visibly pushdown to counting or Turing is when dispatch can inspect more than the top of a stack, and that is precisely the condition where the closure cannot close. If our technique unexpectedly diverges, you have found a weird machine. If it converges, you may have still found a weird machine—but if so, it is a fully specified weird machine, and you have a bisimulation of it. If convergence is not reached within the iteration bound, you get a portion of the equivalence class set modeled, not missing branches, but having not yet folded all the logic into the fused LTS. The proof is mechanized in Lean 4 with zero `sorry` and zero admitted axioms.

It is natural to wonder whether results like the halting problem or Rice's theorem apply to our system. Rice does not apply, for several reasons. Rice's result is only about recursively enumerable programs—it is about the impossibility of a decision algorithm for all recursively enumerable sets. We are not trying to analyze all possible programs or have a general process for all programs; our technique is precisely for the subset with finite dispatch automata, and we are doing extraction, not decision. As for the halting problem: we extract the operational semantics of Turing-complete languages, but we conjecture that no finite binary has a Turing-complete dispatch automaton (Section III). Our system would only diverge on an infinitely large binary—at which point many things would fail regardless.

IV. Method

We take an ELF binary and use Pyvex [12] to lift it to VEX IR. After that point, everything is in Lean with a pretty decent amount of proofs around things. Our work is heavily based on the compositional symbolic execution semantics by Voogd, Laarman, and Lööw [13]; we have ported some of their lemmas to Lean and used their abstraction as our core data structure. A *symbolic branch*—a substitution paired with a path condition guard, following Voogd et al.'s formulation—is the atom of our analysis.

We symbolically execute functions along a weak topological ordering of the Bourdoncle [14] variety. This means that we symbolically execute leaf functions before the functions that call them, and so we can use the symbolic summaries of the callees when we get to the call sites. We have what we call the *dispatch body*, which is basically just a pile of all the symbolic branches from all the functions under analysis—roughly one big switch statement.

We start at the leaves. When we process a leaf, we look at what is on the frontier from that node outward, and we keep checking to see if it composes—is the next symbolic branch reachable from the one we are at? If it is, we compose the two, and if not, we discard the irrelevant branch. At each step we compute the *PC-signature* of the resulting state: a dictionary that maps each guard in the dispatch closure to its three-valued evaluation (true, false, or unknown) in the current abstract state. This signature is the key for our equivalence

classes—two states with the same signature behave identically with respect to which symbolic branches they can take next. We stop iteration when a full pass through composing the dispatch body with itself ceases to yield new signatures. This is the fixpoint of the accumulation of PC-signatures.

At various points we do various simplifications: we strip provably non-aliasing stores in a region-aware way, which really helped with the tractability of the export in terms of size. We also use CVC5 [11] to simplify our guards and update expressions in the emitted LTS, and we reference-count any recurring expressions and turn them into a DAG of references for subexpressions.

Our application of Paige-Tarjan partition refinement [5] starts with the PC-signature classes. We split by transition structure: if a transition from one class leads to different target classes for different members, we split. It is an $O(m \log n)$ algorithm, so plenty performant. In practice, the PC-signature classes almost always already are the bisimulation quotient—the refinement step rarely needs to split further.

Pyvex is a key trust boundary for us; it feels like a sensible enough thing to rely on. A collaborator is working on a PDB variant of our system—the goal was always to be underlying IR agnostic, we have just not built for multiple at once yet. We require non-position-independent ELF binaries at present; it may be possible to expand to position-independent executables, the only issue being that we need the memory region information from ELF headers. So far, we have only run on binaries compiled with zero optimization.

A. Worked Example

TODO: A complete walkthrough of a toy two-function parser (e.g., `parse_expr` dispatching on token type to `parse_atom`, which may recurse). Show: (1) the VEX IR blocks extracted by pyvex, (2) the symbolic branches produced by `blockToCompTree`, (3) one round of composition with PC-signature computation, (4) convergence after two rounds, (5) the final LTS with its guards and transitions, (6) the Soufflé facts derived from it.

B. Algorithm

TODO: Pseudocode listing for the main pipeline: WTO ordering $\rightarrow$ per-function symbolic execution $\rightarrow$ composition fixpoint $\rightarrow$ PC-signature convergence $\rightarrow$ Paige-Tarjan refinement $\rightarrow$ LTS extraction.

V. Evaluation

We have run the pipeline on 13 targets across three domains: parsers (tinyc, mjs, cjson, parson, json, lisp, calc, simplearithmeticparser, cgi_decode), interpreters (QuickJS JS_CallInternal, Lua luaV_execute), and terminal emulators (libtsm, st). All 13 converge.

We have done roughly three kinds of evaluation. The first was an LLM-assisted review of the extracted LTS for libtsm, where we passed a description of terminal emulator vulnerability classes from Leadbeater's survey [15] along with the extracted LTS to an LLM agent. The agent was able to confirm that libtsm does not implement the features (title reporting, DECRQSS, font query) that would make it susceptible to echoback vulnerabilities. We also confirmed some bugs that are not vulnerabilities in this case, but it is of note that we found those bugs.

The second and third evaluations concerned CVE confirmation on mjs v1.20.1, which has ten known parser stack overflow vulnerabilities (CVE-2020-36366 through CVE-2020-36375) in its recursive descent parser functions. We extracted the dispatch automaton from the vulnerable binary, anonymized all function and state names, and converted the LTS to Soufflé [6] fact files. In the final version we got to a blind setup where the agent was only given toy sets of sample facts—one vulnerable (mutual recursion with token-type guards, no depth bound) and one safe (mutual recursion with a depth counter that is tested and decremented)—that would exhibit the kinds of behavior for the kind of vulnerability the agent was asked to try to detect. The agent wrote a 119-line Soufflé program that distinguished bounded from unbounded recursion by checking for zero-test base cases and decrement symbols. When applied to the real anonymized facts, it correctly flagged `parse_block`, `parse_statement`, `parse_statement_list`, `parse_value`, `parse_array`, `parse_object`, and `parse_pair`—all corresponding to the known CVEs—plus `gc_mark` and `gc_mark_object`, a garbage collector recursion cycle.

The key finding: structural cycle detection alone—the kind available from existing tools like Ddisasm [16]—finds mutual recursion but cannot distinguish bounded from unbounded. Our symbolic guard expressions enable the depth-bound heuristic that makes the distinction. This is the gap we fill.

VI. Related Work

Ddisasm [16] is a Soufflé-based disassembler that gives you structural but not semantic Datalog facts—there are no symbolic guards, so you are not getting the branching predicates, at most structure. Formulog [17] does Datalog with SMT over source code, but that means you have got to have source and a model for that language.

STALAGMITE [18] was one of our key inspirations. They recover grammars, working over LLVM bitcode, but they do have to modify the source code to get their instrumentation into the binary. Theirs is a more intrusive analysis, but it is really good work. What they do is sort of fuzzing to detect SMT relations that are the dispatch structure of the grammar, but theirs is an approximate approach, whereas ours is complete.

Vaandrager et al. [19] and Frohme and Steffen [20] do automata learning. We do not use a learning approach—we do static analysis over the binary itself. Daikon [21] infers likely value invariants from traces; we extract the exact control-flow structure from the binary. LibLISA [22] works at the level of inferring the semantics of fragments of an ISA at a per-CPU-variant level of precision—it is one level lower down the stack than us, and complementary.

Our work is heavily based on the compositional symbolic execution semantics by Voogd, Laarman, and Lööw [13]. Theirs is a Coq mechanization proving soundness and completeness of symbolic execution. We have ported some of their lemmas to Lean and used their abstraction as our core data structure. Without them, we could not prove that the weak topological ordering was both precise and complete.

VII. Conclusion

We have presented a technique for extracting verified dispatch automata from ELF binaries. The extracted LTS is bisimilar to the concrete dispatch behavior, with proofs mechanized in Lean 4. The Datalog-queryable output enables automated vulnerability detection that was previously not possible from binary analysis alone, because no existing tool exports symbolic guard expressions as queryable facts.

References

[1] Python Software Foundation, "The Python language reference — introduction," https://docs.python.org/3/reference/introduction.html, 2026, accessed April 2026.

[2] P. E. Metzger, "Slow but steady: Achieving real security within two decades," Keynote, Language-Theoretic Security Workshop (LangSec), IEEE Security and Privacy Workshops, 2017.

[3] L. Sassaman, M. L. Patterson, and S. Bratus, "Security applications of formal language theory," *IEEE Security & Privacy*, vol. 11, no. 3, pp. 52–59, 2013.

[4] T. A. Henzinger, R. Majumdar, and J.-F. Raskin, "A classification of symbolic transition systems," in *Proceedings of STACS 2000*, ser. Lecture Notes in Computer Science, vol. 1770. Springer, 2000, pp. 13–34.

[5] R. Paige and R. E. Tarjan, "Three partition refinement algorithms," *SIAM Journal on Computing*, vol. 16, no. 6, pp. 973–989, 1987.

[6] H. Jordan, B. Scholz, and P. Subotić, "Soufflé: On synthesis of program analyzers," in *Computer Aided Verification (CAV)*, ser. Lecture Notes in Computer Science, vol. 9780. Springer, 2016, pp. 422–430.

[7] T. A. Henzinger, R. Jhala, R. Majumdar, and G. Sutre, "Lazy abstraction," in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2002, pp. 58–70.

[8] R. Alur and P. Madhusudan, "Visibly pushdown languages," in *Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 2004, pp. 202–211.

[9] R. Milner, *Communication and Concurrency*. Prentice Hall, 1989.

[10] R. J. van Glabbeek, "The linear time – branching time spectrum," in *CONCUR '90: Theories of Concurrency: Unification and Extension*, ser. Lecture Notes in Computer Science, vol. 458. Springer, 1990, pp. 278–297.

[11] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar, "cvc5: A versatile and industrial-strength SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. Lecture Notes in Computer Science, vol. 13243. Springer, 2022, pp. 415–442.

[12] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SOK: (state of) the art of war: Offensive techniques in binary analysis," in *IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 138–157.

[13] E. Voogd, Å. A. A. Kløvstad, E. B. Johnsen, and A. Wąsowski, "Compositional symbolic execution semantics," *Theoretical Computer Science*, vol. 1044, p. 115263, 2025.

[14] F. Bourdoncle, "Efficient chaotic iteration strategies with widenings," in *Formal Methods in Programming and Their Applications*, ser. Lecture Notes in Computer Science, vol. 735. Springer, 1993, pp. 128–141.

[15] D. Leadbeater, "ANSI terminal security," https://dgl.cx/2023/09/ansi-terminal-security, 2023.

[16] A. Flores-Montoya and E. Schulte, "Datalog disassembly," in *29th USENIX Security Symposium*. USENIX Association, 2020, pp. 1075–1092.

[17] A. Bembenek, M. Greenberg, and S. Chong, "Formulog: Datalog for SMT-based static analysis," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–31, 2020.

[18] L. Bettscheider and A. Zeller, "Look ma, no input samples! Mining input grammars from code with symbolic parsing," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*. ACM, 2024, pp. 333–337.

[19] F. Vaandrager, B. Garhewal, J. Rot, and T. Wissmann, "A new myhill-nerode theorem for register automata and nominal automata," *Information and Computation*, vol. 294, p. 105072, 2023.

[20] M. Frohme and B. Steffen, "Compositional learning of mutually recursive procedural systems," *International Journal on Software Tools for Technology Transfer*, vol. 23, pp. 521–543, 2021.

[21] M. D. Ernst, J. Cockrell, W. G. Griswold, and D. Notkin, "Dynamically discovering likely program invariants to support program evolution," *IEEE Transactions on Software Engineering*, vol. 27, no. 2, pp. 99–123, 2001.

[22] J. Craaijo, F. Verbeek, and B. Ravindran, "libLISA: Instruction discovery and analysis on x86-64," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA2, pp. 1–29, 2024.